

Руководство тестировщика

Med Doctor Assignment Service. Версия документации: 2026-04-29.

Содержание

Руководство тестировщика

1. Назначение документа
2. Что считается успешной работой
3. Подготовка тестового стенда
4. Функциональные сценарии
 - TC-01. Старт сервиса при доступной Orchestra
 - TC-02. Старт сервиса при недоступной Orchestra
 - TC-03. Polling видит визиты в очереди «Врач не назначен»
 - TC-04. Dry-run не изменяет состояние визита
 - TC-05. Корреляция посадки врача
 - TC-06. `SERVICE_POINT_OPEN` не запускает mutating workflow
 - TC-07. Локальный выбор услуги без med-robot
 - TC-08. med-robot в режиме JSON-массива услуг
 - TC-09. med-robot в режиме номера талона
 - TC-10. Ошибка med-robot с fallback
 - TC-11. Ошибка med-robot со skip
 - TC-12. Реальное назначение одного визита
 - TC-13. Recheck перед transfer
 - TC-14. Повторный маршрутный шаг того же визита
 - TC-15. Дедупликация того же маршрутного шага
 - TC-16. Ошибка assign `INACTIVE`
 - TC-17. Ошибка assign `NO_STARTED_SERVICE_POINT_SESSION`
 - TC-18. Неверный source entry point id
5. Негативные проверки конфигурации
6. Проверки нагрузки на стенде
7. Что фиксировать в тестовом протоколе
8. Критерии приемки

1. Назначение документа

Документ описывает функциональные, интеграционные и негативные проверки Med Doctor Assignment Service. Основной объект тестирования - корректность автоматического назначения визита из очереди «Врач не назначен» в зависимости от событий Orchestra, рабочего профиля врача, состояния визита, результата med-robot и защитных настроек.

2. Что считается успешной работой

Сервис работает корректно, если:

- не падает при временной недоступности Orchestra;
- прогревает branch cache;
- видит визиты в очереди «Врач не назначен»;
- корректно восстанавливает врача по `USER_SERVICE_POINT_SESSION_START -> SET_WORK_PROFILE` ;
- не запускает опасные мутации по преждевременному `SERVICE_POINT_OPEN` ;
- вызывает med-robot в нужном режиме;
- при `dry-run=true` не меняет состояние Orchestra;
- при `dry-run=false` назначает услугу и переводит визит в ожидаемую очередь;
- не обрабатывает один и тот же маршрутный шаг повторно в пределах TTL;
- допускает повторную обработку того же `visitId` , если начался новый маршрутный шаг;
- корректно останавливает цикл после контекстной ошибки `mutation` .

3. Подготовка тестового стенда

Перед тестами должны быть известны:

Параметр	Значение
URL Orchestra	заполнить в протоколе тестирования.
Branch id	тестовое отделение.
Queue id «Врач не назначен»	фактический queue id.
Source entry point id	значение для fromId .
Staff id врача	тестовый врач.
Service point id	тестовая точка обслуживания.
Work profile id	профиль с нужными очередями.
URL med-robot	если участвует в сценариях.
Номер тестового талона	для med-robot plain text.
Visit id	для сверки логов.

Первый тестовый прогон выполняется с:

```
application:
  assignment:
    dry-run: true
    max-visits-per-cycle: 3
    allowed-branches: [1]
  websocket:
    enabled: false
  med-robot:
    enabled: false
```

4. Функциональные сценарии

ТС-01. Старт сервиса при доступной Orchestra

Предусловия: Orchestra доступна, учетная запись корректна. **Шаги:** запустить сервис.

Ожидаемо: сервис стартует, выполняет cache refresh, не пишет критических ошибок.

Контрольные строки:

```
Start cache refresh for branch
Finish cache refresh for branch
```

ТС-02. Старт сервиса при недоступной Orchestra

Предусловия: Orchestra выключена или недоступна по сети. **Шаги:** запустить сервис.

Ожидаемо: процесс не завершается аварийно; лог содержит retry.

Контрольные строки:

```
Service will stay alive and retry later
Schedule Orchestra REST reconnect/cache bootstrap retry
```

ТС-03. Polling видит визиты в очереди «Врач не назначен»

Предусловия: `polling-enabled=true`, в очереди есть тестовый визит. **Шаги:** дождаться запуска `polling`. **Ожидаемо:** лог содержит количество визитов и попытку анализа маршрута.

ТС-04. Dry-run не изменяет состояние визита

Предусловия: `dry-run=true`, есть подходящий визит. **Шаги:** дождаться `assignment cycle`. **Ожидаемо:** в логах есть план назначения, но состояние визита в Orchestra не изменилось.

ТС-05. Корреляция посадки врача

Предусловия: `websocket.enabled=true`, врач входит на точку обслуживания. **Шаги:** выполнить посадку врача в интерфейсе Orchestra. **Ожидаемо:** сервис получает `USER_SERVICE_POINT_SESSION_START`, затем `SET_WORK_PROFILE`, после чего запускает цикл с контекстом врача.

ТС-06. SERVICE_POINT_OPEN не запускает mutating workflow

Предусловия: `service-point-open-trigger-enabled=false`. **Шаги:** вызвать сценарий, при котором публикуется `SERVICE_POINT_OPEN`. **Ожидаемо:** событие может быть залогировано, но реальный `assignment cycle` по этому `trigger` не выполняется.

ТС-07. Локальный выбор услуги без med-robot

Предусловия: `med-robot.enabled=false`, маршрут визита содержит услугу, доступную врачу. **Шаги:** запустить цикл. **Ожидаемо:** сервис выбирает услугу по пересечению маршрута визита и услуг врача.

ТС-08. med-robot в режиме JSON-массива услуг

Предусловия: `request-body-mode=UNSERVED_SERVICE_IDS_JSON_ARRAY`, у визита есть `unservedVisitServices`. **Шаги:** запустить цикл. **Ожидаемо:** `med-robot` получает JSON-массив `id` непройденных услуг, возвращает `serviceId/queueId`.

ТС-09. med-robot в режиме номера талона

Предусловия: `request-body-mode=TICKET_NUMBER_PLAIN_TEXT`, у визита есть номер талона. **Шаги:** запустить цикл. **Ожидаемо:** `body` запроса к `med-robot` содержит текстовый номер талона, даже если `unservedVisitServices` пустой.

ТС-10. Ошибка med-robot с fallback

Предусловия: `med-robot.enabled=true`, `error-handling-mode=FALLBACK_TO_LOCAL`, `med-robot` недоступен. **Шаги:** запустить цикл. **Ожидаемо:** сервис пишет ошибку `med-robot` и пытается выбрать услугу локально.

ТС-11. Ошибка med-robot со skip

Предусловия: `error-handling-mode=SKIP_VISIT`. **Шаги:** med-robot возвращает 500 или недоступен. **Ожидаемо:** текущий визит пропускается в этом цикле без локального назначения.

ТС-12. Реальное назначение одного визита

Предусловия: все dry-run проверки успешны, `dry-run=false`, `max-visits-per-cycle=1`. **Шаги:** запустить цикл. **Ожидаемо:** один визит получает выбранную услугу и переводится в целевую очередь.

ТС-13. Recheck перед transfer

Предусловия: `recheck-visit-before-transfer=true`. **Шаги:** между выбором и transfer изменить состояние визита вручную или другим процессом. **Ожидаемо:** сервис пересчитывает визит и не выполняет некорректный transfer.

ТС-14. Повторный маршрутный шаг того же визита

Предусловия: визит после завершения услуги снова попадает в «Врач не назначен» с новым `currentVisitService.id`. **Шаги:** пройти одну услугу, затем вернуть визит в очередь. **Ожидаемо:** сервис допускает повторную обработку, потому что fingerprint изменился.

ТС-15. Дедупликация того же маршрутного шага

Предусловия: один и тот же маршрутный шаг порождает повторные события. **Шаги:** вызвать повторные события в пределах `processed-visit-ttl-seconds`. **Ожидаемо:** повторная обработка блокируется.

ТС-16. Ошибка assign INACTIVE

Предусловия: Orchestra возвращает `userState=INACTIVE`. **Шаги:** запустить реальный цикл. **Ожидаемо:** сервис считает assign контекстной ошибкой и не делает transfer в том же цикле.

ТС-17. Ошибка assign NO_STARTED_SERVICE_POINT_SESSION

Предусловия: Orchestra возвращает `NO_STARTED_SERVICE_POINT_SESSION`. **Шаги:** запустить цикл до стабилизации сессии. **Ожидаемо:** цикл останавливается без transfer.

ТС-18. Неверный source entry point id

Предусловия: в тестовом профиле намеренно указан неверный `source-entry-point-id-by-branch`. **Шаги:** запустить реальный цикл только на тестовом визите. **Ожидаемо:** transfer падает предсказуемо, ошибка содержит контекст; цикл не продолжает массовые мутации.

5. Негативные проверки конфигурации

Проверка	Ожидаемо
неверный <code>unknown-doctor-queue-id</code>	сервис не находит визиты, пишет пустую очередь.
пустой <code>allowed-branches</code> на стенде	рассматривать как риск: все отделения разрешены.
<code>dry-run=false</code> без <code>source entry point id</code>	<code>transfer</code> невозможен или небезопасен; тест должен быть заблокирован.
<code>websocket.enabled=true</code> , но <code>/qprevents/events</code> недоступен	сервис продолжает <code>polling</code> .
<code>med-robot.enabled=true</code> , URL неверный	<code>fallback/skip</code> согласно настройке.
<code>require-known-queue=true</code> , <code>med-robot</code> вернул неизвестную очередь	результат <code>med-robot</code> не применяется.
<code>abort-cycle-on-forbidden-mutation=true</code> , первый 403	цикл останавливается.

6. Проверки нагрузки на стенде

Для тестового стенда не нужно имитировать `production`-нагрузку, но нужно убедиться, что сервис не создает лишний шум:

- `max-visits-per-cycle` ограничивает число визитов за цикл;
- `polling-cron` не должен запускать цикл слишком часто без необходимости;
- `stale-cache-duration-seconds` не должен быть слишком маленьким;
- `repeated events` блокируются `event-deduplication-ttl-seconds`;
- `repeated route step` блокируется `processed-visit-ttl-seconds`;
- после первого 403/контекстной ошибки цикл не продолжает десятки одинаковых запросов.

7. Что фиксировать в тестовом протоколе

- версия сборки;
- дата и время теста;
- конфигурационный профиль;
- `dry-run` `true/false`;
- `branch id`;
- `queue id`;
- `source entry point id`;
- `staff/service point/work profile`;
- `visit id` и номер талона;
- выбранный `trigger`;

- выбранный `serviceId` и `queueId`;
- был ли `med-robot` вызван;
- статус `assign`;
- статус `transfer`;
- итоговое состояние визита в `Orchestra`;
- ссылки на фрагменты логов.

8. Критерии приемки

- [] Все обязательные `smoke tests` пройдены.
- [] `Dry-run` сценарии не меняют состояние `Orchestra`.
- [] `Websocket`-корреляция работает или документирован `fallback` на `polling`.
- [] `med-robot` проверен в требуемом режиме.
- [] Реальная мутация проверена на одном тестовом визите.
- [] Защитные настройки оставлены включенными.
- [] Нет массовых `403/500` в логах.
- [] Ошибки `med-robot` и `Orchestra` обрабатываются предсказуемо.
- [] Повторная обработка маршрутных шагов работает корректно.
- [] Протокол передан внедренцу и техподдержке.